CS 200 Project 3: Team 5 Requirement Report, Fall 2026

Header Table:

Team Member Name	CWID Number	Crimson Email Address	Hours Contributed
Sarah Rechkemmer	12395321	sgrechkemmer@crimson.ua.edu	4
Ross Craig	12305075	rlcraig2@crimson.ua.edu	4
Isaac Kim	12192384	ikim15@crimson.ua.edu	4
Jackson Turner	12354283	jturner29@crimson.ua.edu	4
For Project 3, Ross Craig will submit the final report.			

Task Distribution Table:

Team Member Name	Tasks Performed	Percentage Contributed
Sarah Rechkemmer	UML Class Diagram	25%
Ross Craig	Discussion	25%
Isaac Kim	Diagram Review	25%
Jackson Turner	UML Sequence Diagrams	25%

Task 1: UML Class Diagram

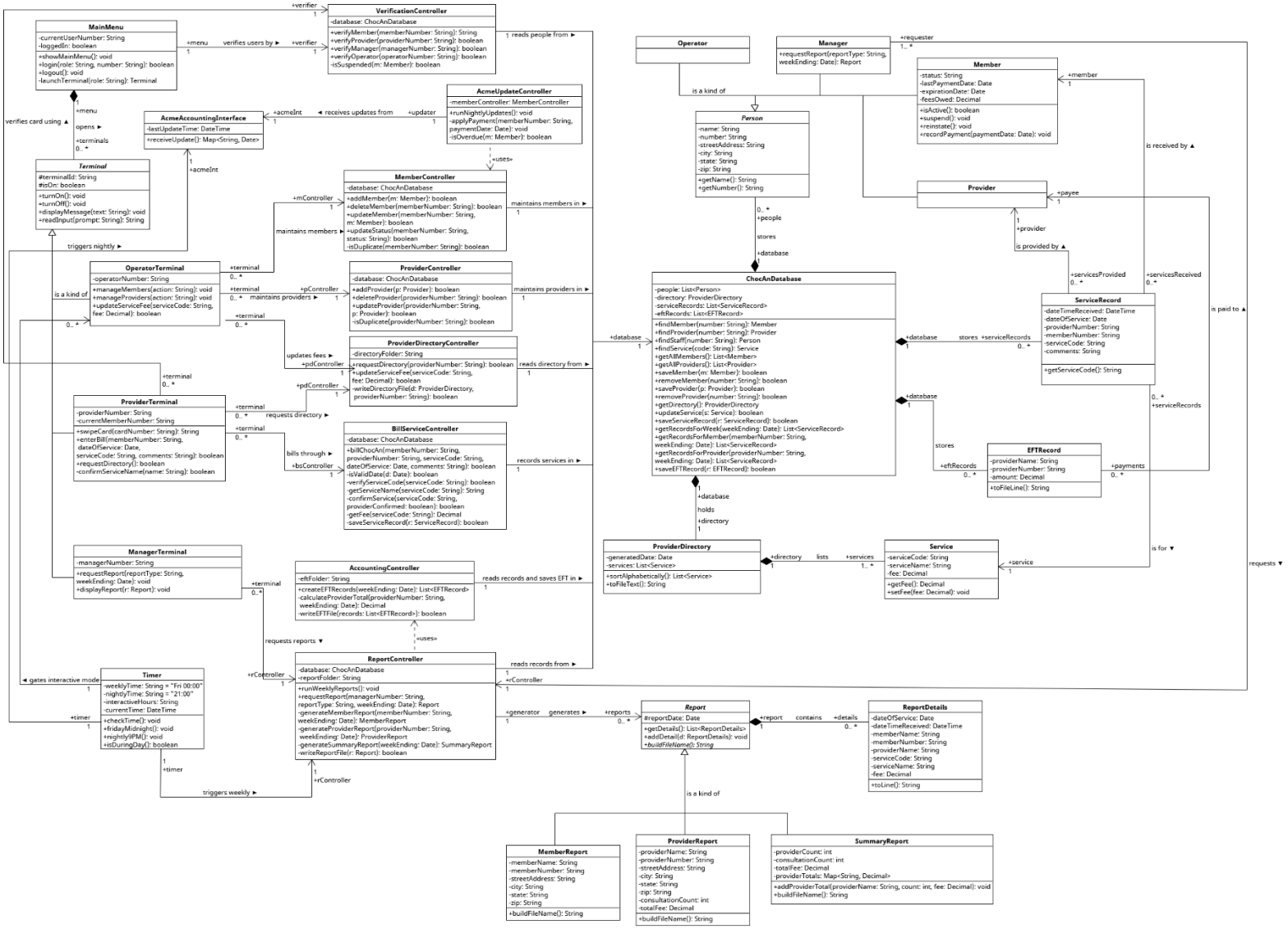


Figure 1. UML Class Diagram

Task 2: UML Sequence Diagrams

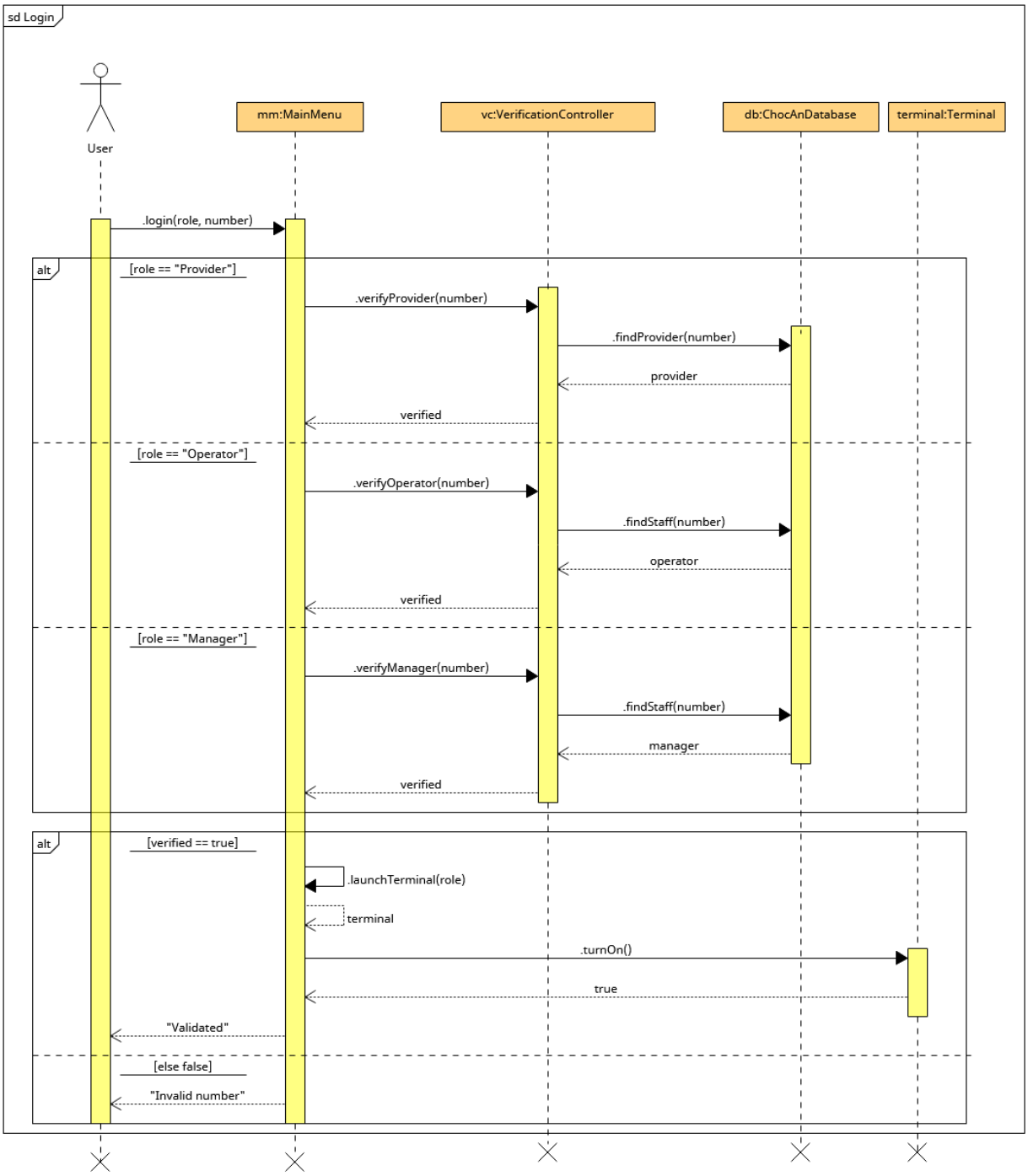


Figure 2. Login Reference Sequence Diagram

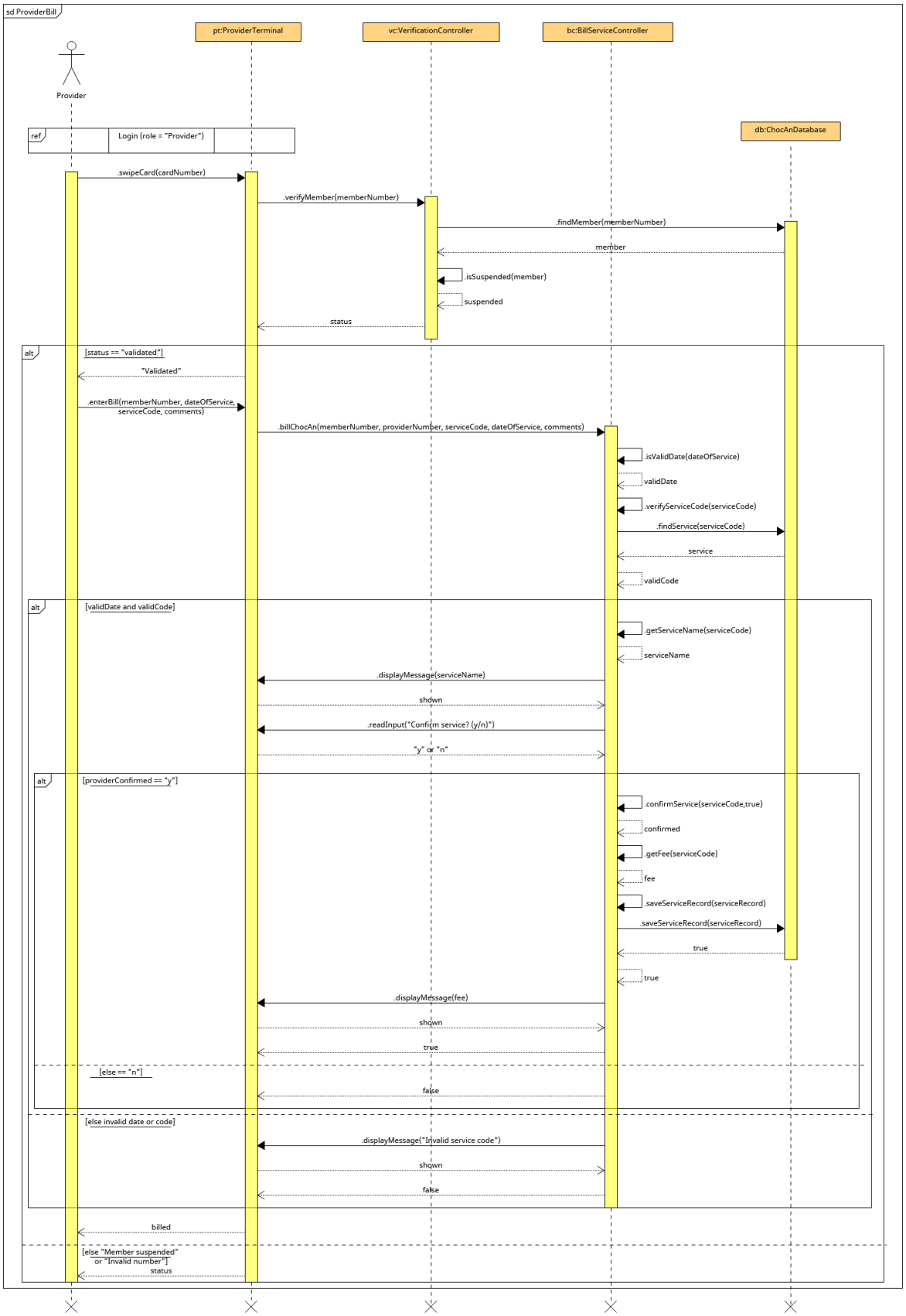

sd ProviderBill
pt:ProviderTerminal
vc:VerificationController
bc:BillServiceController
db:ChocAnDatabase
Provider
ref
Login (role = "Provider")
.swipeCard(cardNumber)
.verifyMember(memberNumber)
.findMember(memberNumber)
member
.isSuspended(member)
suspended
status
alt
[status == "validated"]
"Validated"
.enterBill(memberNumber, dateOfService, serviceCode, comments)
.billChocAn(memberNumber, providerNumber, serviceCode, dateOfService, comments)
.isValidDate(dateOfService)
validDate
.verifyServiceCode(serviceCode)
.findService(serviceCode)
service
validCode
alt
[validDate and validCode]
.getServiceName(serviceCode)
serviceName
.displayMessage(serviceName)
shown
.readInput("Confirm service? (y/n)")
"y" or "n"
alt
[providerConfirmed == "y"]
.confirmService(serviceCode,true)
confirmed
.getFee(serviceCode)
fee
.saveServiceRecord(serviceRecord)
.saveServiceRecord(serviceRecord)
true
true
.displayMessage(fee)
shown
true
[else == "n"]
false
[else invalid date or code]
.displayMessage("Invalid service code")
shown
false
billed
[else "Member suspended" or "Invalid number"]
status

Figure 3. Provider Billing Sequence Diagram.

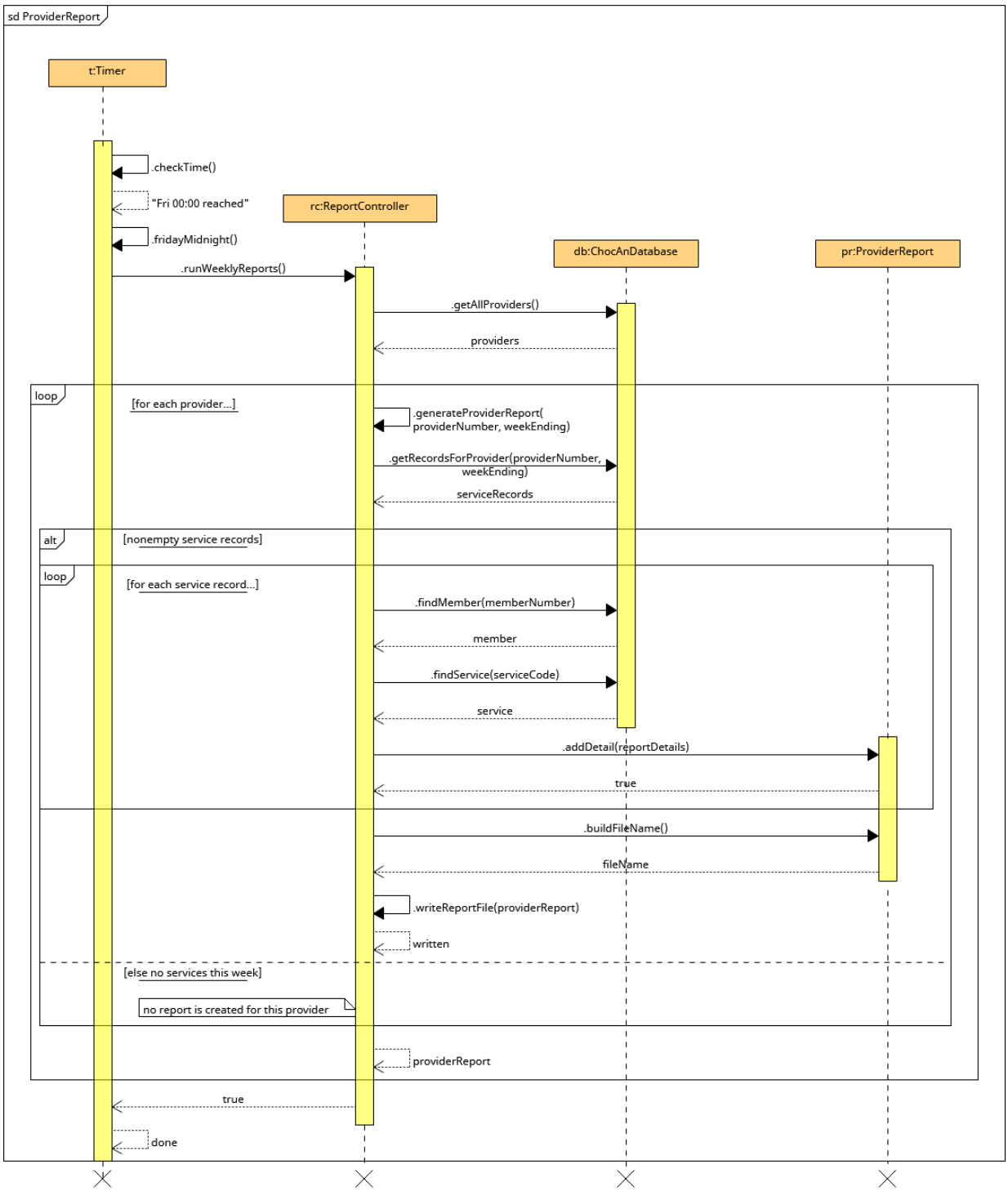


Figure 4. Provider Report Sequence Diagram.

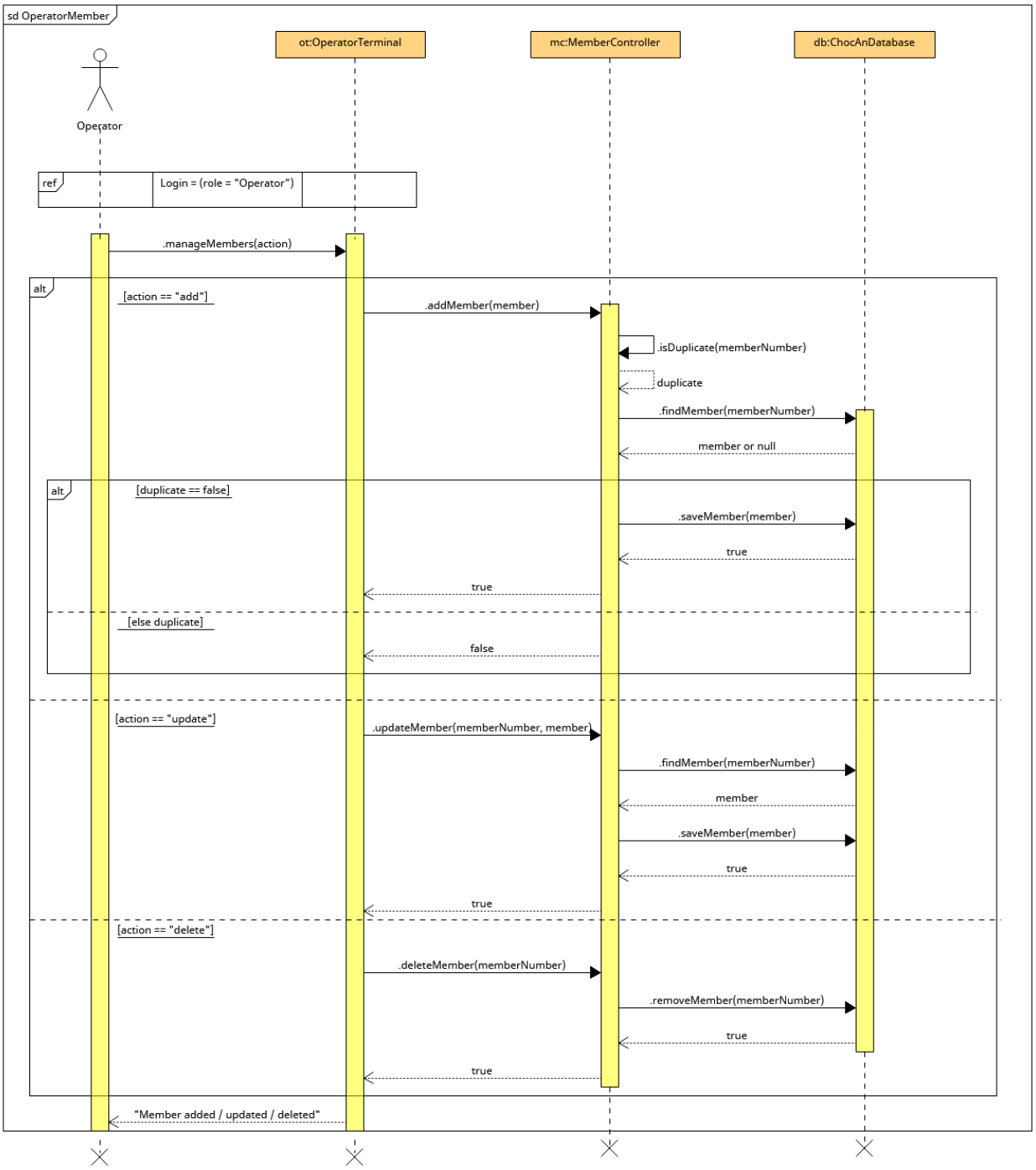


Figure 5. Operator Member Sequence Diagram.

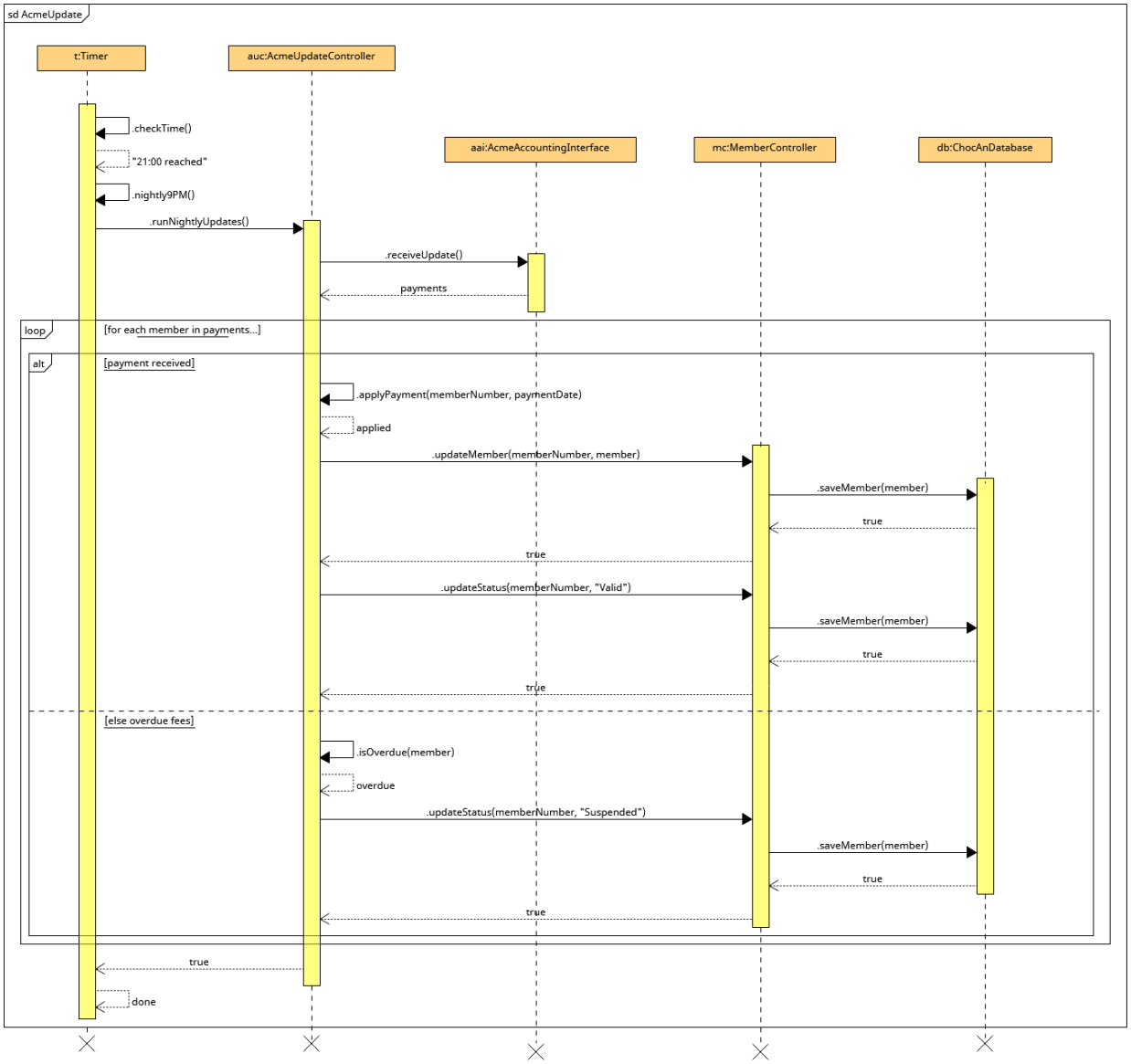


Figure 6. Acme Update Sequence Diagram.

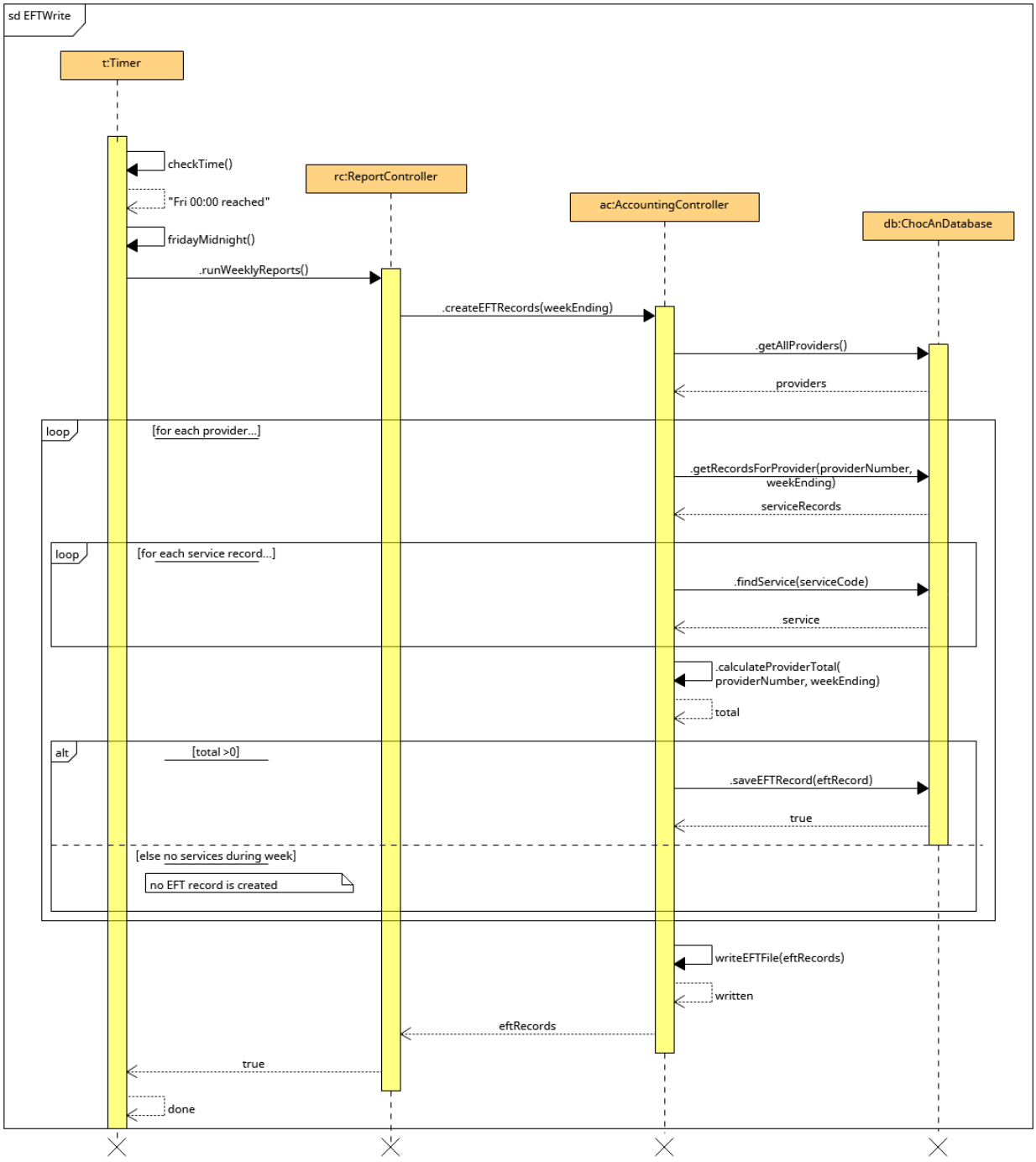


Figure 7. EFT Records Sequence Diagram.

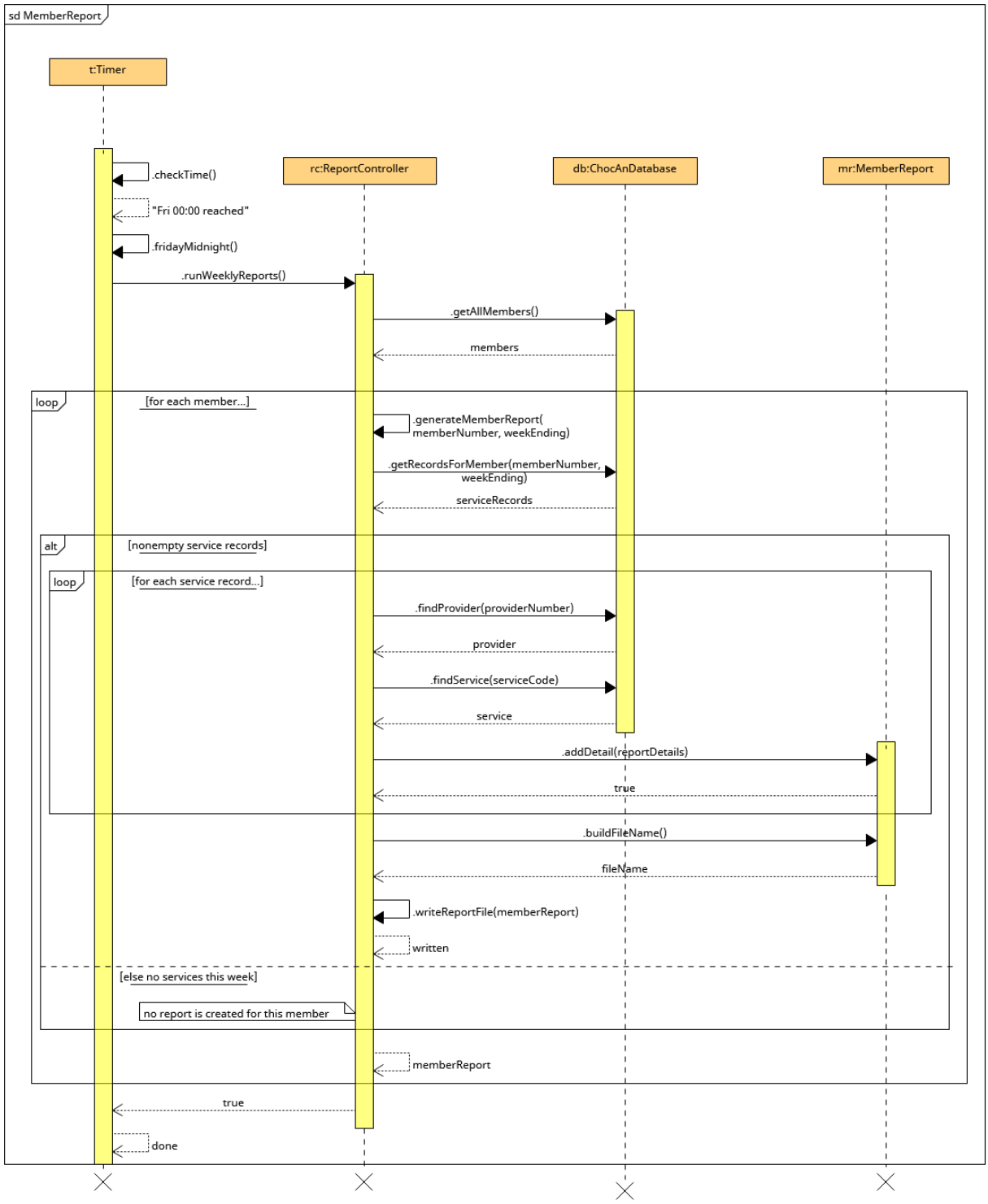


Figure 8. Member Report Sequence Diagram.

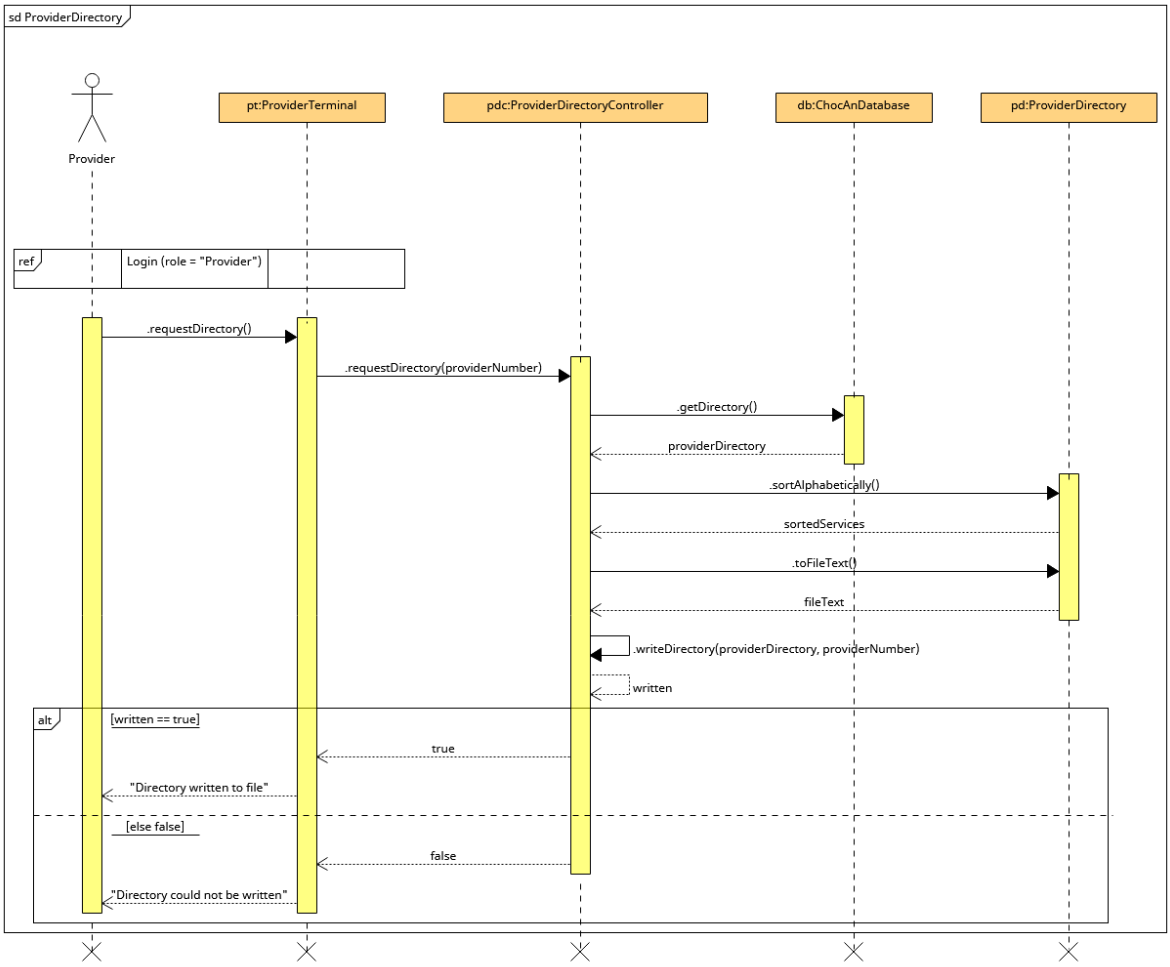


Figure 9. Provider Directory Sequence Diagram.

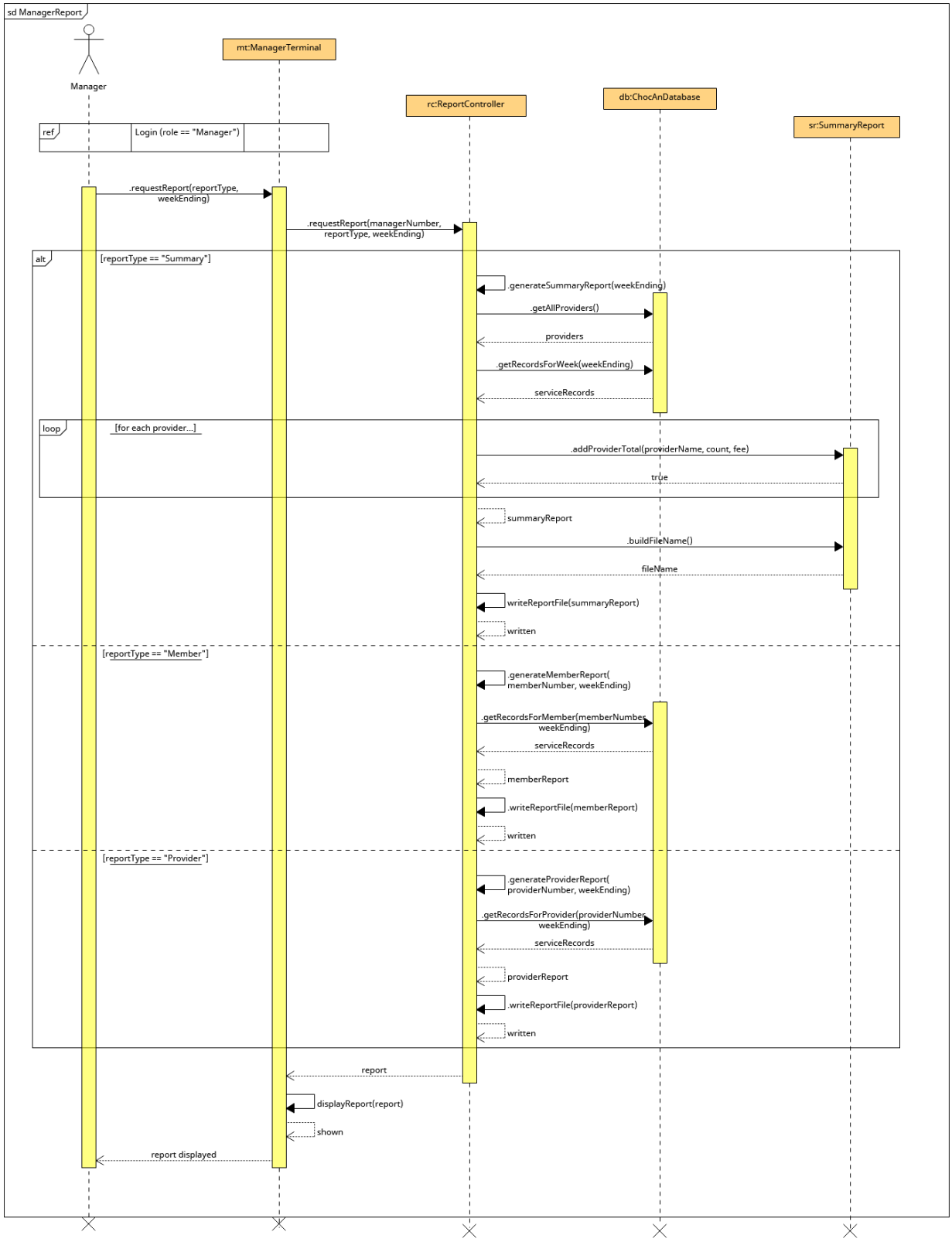


Figure 10. Manager Report Sequence Diagram.

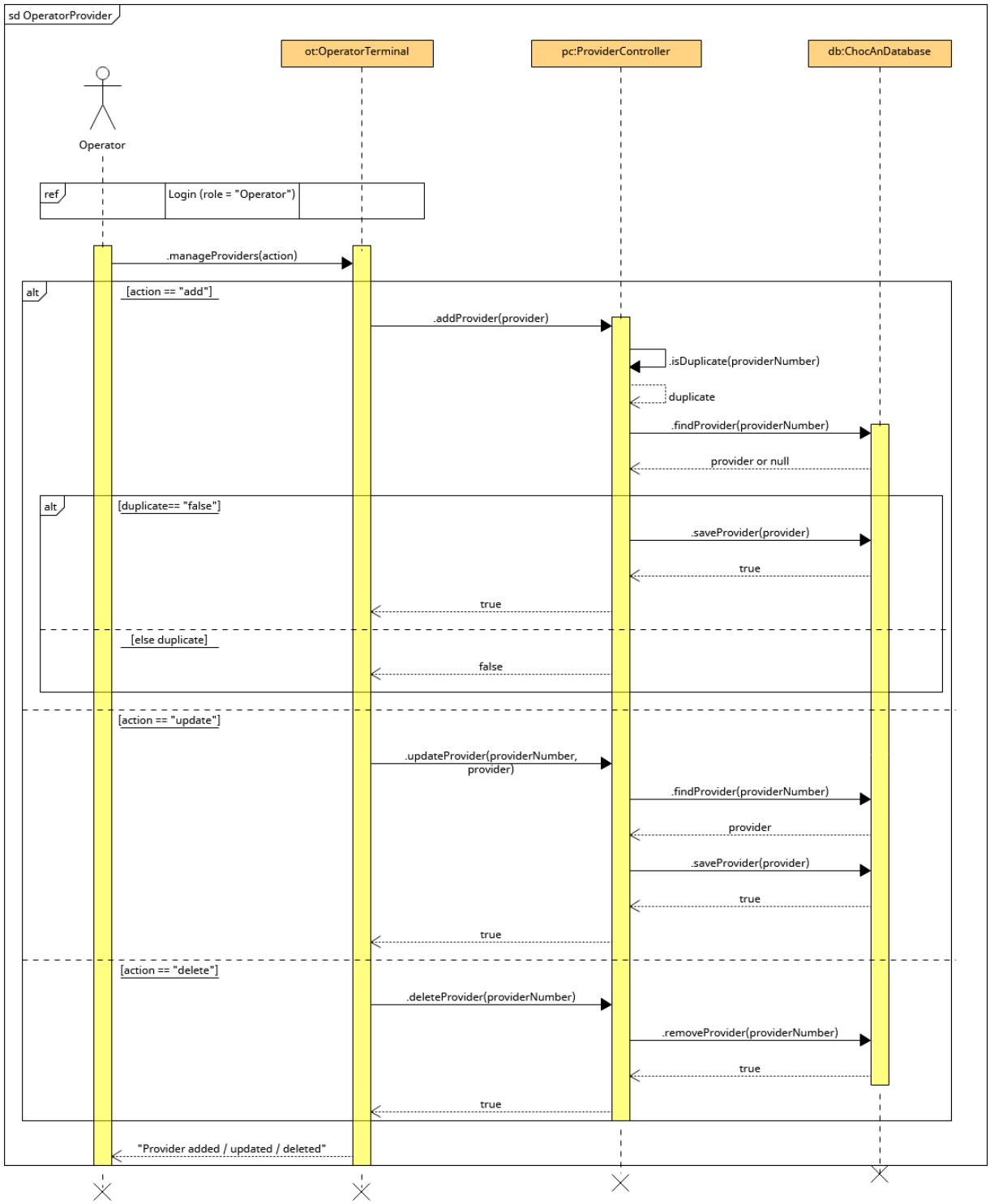


Figure 11. Operator Provider Sequence Diagram.

Task 3: Discussion

For the cohesion of our solution, a class such as the MemberController class is very cohesive. Its methods only serve to maintain member records and nothing else, making it an extremely cohesive module. Our other controllers are also very cohesive, as each one is responsible for a single task. VerificationController only verifies users, BillServiceController only handles the billing of a service, ProviderDirectoryController only produces the provider directory, AccountingController only creates the EFT records, and ReportController only generates the reports. Then, the Timer is coupled to three classes, being the OperatorTerminal, AcmeAcountingInterface, and ReportController, for initiating the actions of those classes. For the coupling of our system, there is one highly coupled class, the ChocAnDatabase. This is because many of our design coupled choices directly interact with or are contained in the ChocAnDatabase, creating a point of very high coupling. The rest of our system remains lowly coupled as the terminals depend only on the specific controllers they need, and the only controller-to-controller dependencies are the two <<uses>> relationships for AcmeUpdateController to MemberController, and ReportController to AccountingController. We maximized the cohesion of the system by ensuring that if a function changed member status, it was put inside the MemberController and vice versa. This ensures that the cohesion stays high, as the AccountingController will never affect member status, and even the nightly Acme update goes through the MemberController's updateStatus method.

We mitigated the coupling of our system by reducing dependencies wherever we thought possible. Unfortunately, for the ChocAnDatabase we did not see a way to get rid of all the coupling, so we kept it as a data-access class only with no business actions, which keeps it from becoming a "god object". This leaves the terminal and report classes, which are minimally coupled to the other classes that perform display and data functions. For cohesion and coupling, these principles help maintain the system because an update to one responsibility only requires a change to one controller. For flexibility, a system with high cohesion and low coupling is easier to extend. For example, a new report type only needs a new subclass of Report and one generate method in the ReportController, and a new terminal only needs a new subclass of Terminal. If there were high coupling, it would be very hard to add a new controller without changing the way various other controllers operate, and in a system with low cohesion, adding a new controller may require many other controllers to be updated. For scalability, keeping these principles in mind will reduce the amount of time spent editing and updating numerous controllers. We would argue that our system fits within these constraints, as each controller only affects the items within its own responsibility, and our solution has very low coupling outside of the ChocAnDatabase, which is the central data hub that many controllers must interact with.